

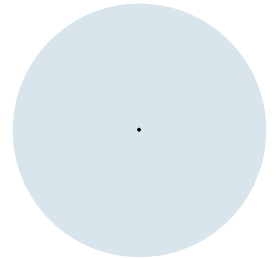
Einführung in die Programmierung (WIN+BIT)

Rekursion

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Rekursion



„Um Rekursion zu verstehen, muss man zuerst
Rekursion verstehen.“

beliebter Informatiker-Witz



Definition

- Ein Methode M heißt **rekursiv**, wenn sie sich direkt (rekursiver Aufruf) oder indirekt (verschränkt rekursiver Aufruf) selbst aufruft.
- Ansonsten heißt M **iterativ**: Methode besteht nur aus Anweisungen, Schleifen und/oder Aufrufen nicht-rekursiver Methoden; es gibt keine (direkten oder indirekten) Selbstaufrufe.

– Beispiele:

```
static void recursiveMethod(int n) {
    if (n > 10) {
        return;
    }
    System.out.println(n);
    recursiveMethod(n + 1);
}

static void iterativeMethod() {
    for (int i = 0; i < 10; i++) { // Schleife
        if (i % 2 == 0) { // if-Anweisung
            // Aufruf nicht-rekursiver Methode println
            System.out.println(i);
        }
    }
}
```



code22.
Recursion

Rekursion ist Lösungsstrategie

Ein rekursiver Ansatz beinhaltet, dass man ein Problem auf ein oder mehrere "kleinere" Probleme der gleichen Art zurückführt.

Beispiel Fakultät:

$$\begin{aligned}4! &= 4 \cdot 3 \cdot 2 \cdot 1 \\5! &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1\end{aligned}$$

also

$$5! = 5 \cdot 4!$$

Zur Berechnung von $5!$ kann ich also die Lösung des kleineren Problem $4!$ benutzen, indem ich die Lösung des kleineren Problems mit 5 multipliziere.

Das gilt auch allgemein:

$$n! = n \cdot (n - 1)! \text{ wenn } n > 1$$

Außerdem ist für gewisse, kleine Probleminstanzen die Lösung bekannt:

$$1! = 1$$

(und auch per definitionem $0! = 1$)

Fakultät (engl. *factorial*)

Es ist also folgender Algorithmus (Berechnungsvorschrift) möglich:

`factorial( $n$ ):`

Gegeben: $n \geq 1$

Gesucht: $n!$

Berechnung:

- Wenn n den Wert 1 hat, ist die Lösung 1.
- Ansonsten berechne den Wert `factorial( $n - 1$ )` und multipliziere das Ergebnis mit n

Fakultät $n = 1$

factorial(1):

Gegeben: 1

Gesucht: 1!

Berechnung:

- Wenn 1 den Wert 1 hat, ist die Lösung 1.
- ~~Ansonsten berechne den Wert factorial(1 - 1) und multipliziere das Ergebnis mit 1~~

Fakultät $n = 5$

factorial(5):

Gegeben: 5

Gesucht: 5!

Berechnung:

- ~~• Wenn 5 den Wert 1 hat, ist die Lösung 1.~~
- Ansonsten berechne den Wert factorial(5 - 1) und multipliziere das Ergebnis mit 5

Java-Code

```
static int factorial(int n) {  
    if (n == 1) {  
        return 1;  
    }  
    return factorial(n - 1) * n;  
}
```

Berechnung:

- Wenn n den Wert 1 hat, ist die Lösung 1.
- Ansonsten berechne den Wert $\text{factorial}(n - 1)$ und multipliziere das Ergebnis mit n



code22.
Factorial

Java-Code

```
static int factorial(int n) {  
    if (n == 1) {  
        return 1;  
    }  
    return factorial(n - 1) * n;  
}
```

Abbruchbedingung (auch Rekursionsanker genannt):
Für bestimmten kleinen Wert endet die Rekursion

Rekursion: factorial ruft sich selbst auf

factorial(5):

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

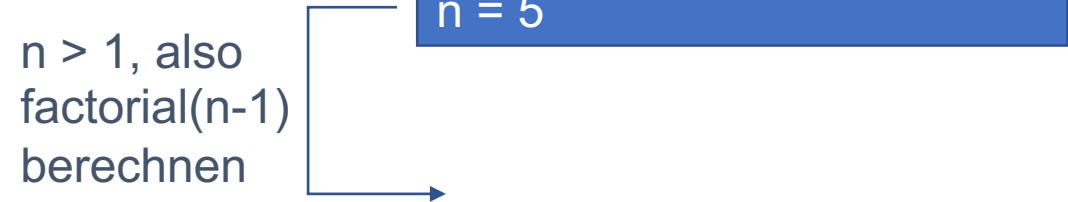
```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```


`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1`, also
`factorial(n-1)`
berechnen



```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1`, also
`factorial(n-1)`
berechnen

zweiter Aufruf der Methode

`n = 4`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

dritter Aufruf der Methode

`n = 3`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

dritter Aufruf der Methode

`n = 3`

`n > 1, also
factorial(n-1)
berechnen`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

dritter Aufruf der Methode

`n = 3`

`n > 1, also
factorial(n-1)
berechnen`

vierter Aufruf der Methode

`n = 2`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

dritter Aufruf der Methode

`n = 3`

`n > 1, also
factorial(n-1)
berechnen`

vierter Aufruf der Methode

`n = 2`

`n > 1, also
factorial(n-1)
berechnen`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

`factorial(5):`

erster Aufruf der Methode

`n = 5`

`n > 1, also
factorial(n-1)
berechnen`

zweiter Aufruf der Methode

`n = 4`

`n > 1, also
factorial(n-1)
berechnen`

dritter Aufruf der Methode

`n = 3`

`n > 1, also
factorial(n-1)
berechnen`

vierter Aufruf der Methode

`n = 2`

`n > 1, also
factorial(n-1)
berechnen`

fünfter Aufruf der Methode

`n = 1`

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```


factorial(5):

erster Aufruf der Methode

n = 5

n > 1, also
factorial(n-1)
berechnen

zweiter Aufruf der Methode

n = 4

n > 1, also
factorial(n-1)
berechnen

dritter Aufruf der Methode

n = 3

n > 1, also
factorial(n-1)
berechnen

vierter Aufruf der Methode

n = 2

n > 1, also
factorial(n-1)
berechnen

fünfter Aufruf der Methode

n = 1

Rückgabe: 1

n = 1, also 1
zurückgeben

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

factorial(5):

erster Aufruf der Methode

n = 5

n > 1, also
factorial(n-1)
berechnen

zweiter Aufruf der Methode

n = 4

n > 1, also
factorial(n-1)
berechnen

dritter Aufruf der Methode

n = 3

n > 1, also
factorial(n-1)
berechnen

vierter Aufruf der Methode

n = 2

Rückgabe: 2

1 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

fünfter Aufruf der Methode

n = 1

Rückgabe: 1

n = 1, also 1
zurückgeben

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

factorial(5):

erster Aufruf der Methode

n = 5

n > 1, also
factorial(n-1)
berechnen

zweiter Aufruf der Methode

n = 4

n > 1, also
factorial(n-1)
berechnen

dritter Aufruf der Methode

n = 3

Rückgabe: 6

2 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

vierter Aufruf der Methode

n = 2

Rückgabe: 2

1 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

fünfter Aufruf der Methode

n = 1

Rückgabe: 1

n = 1, also 1
zurückgeben

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

factorial(5):

$n > 1$, also
 $\text{factorial}(n-1)$
berechnen

erster Aufruf der Methode

$n = 5$

zweiter Aufruf der Methode

$n = 4$

Rückgabe: 24

$6 * n$
zurückgeben

$n > 1$, also
 $\text{factorial}(n-1)$
berechnen

dritter Aufruf der Methode

$n = 3$

Rückgabe: 6

$2 * n$
zurückgeben

$n > 1$, also
 $\text{factorial}(n-1)$
berechnen

vierter Aufruf der Methode

$n = 2$

Rückgabe: 2

$1 * n$
zurückgeben

$n > 1$, also
 $\text{factorial}(n-1)$
berechnen

fünfter Aufruf der Methode

$n = 1$

Rückgabe: 1

$n = 1$, also 1
zurückgeben

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

factorial(5):

n > 1, also
factorial(n-1)
berechnen

erster Aufruf der Methode

n = 5

Rückgabe: 120

24 * n zurückgeben.
Ergebnis ist 120.

zweiter Aufruf der Methode

n = 4

Rückgabe: 24

6 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

dritter Aufruf der Methode

n = 3

Rückgabe: 6

3 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

vierter Aufruf der Methode

n = 2

Rückgabe: 2

1 * n
zurückgeben

n > 1, also
factorial(n-1)
berechnen

fünfter Aufruf der Methode

n = 1

Rückgabe: 1

n = 1, also 1
zurückgeben

```
int factorial(int n) {  
    if (n == 1) { return 1; }  
    return factorial(n-1)*n;  
}
```

Rekursion, Zusammenfassung

- Aufruf einer Methode in ihrer eigenen Implementierung
- Methodenaufruf
 - **Fallunterscheidung** in Implementierung
 - Für **Abbruchbedingung** (Rekursionsanker) festes Ergebnis (z.B. bestimmter Parameterwert)
 - Ansonsten Implementierung/Verarbeitung/Berechnung mittels Aufruf der **gleichen** Methode mit **veränderten Parametern** (meist "kleinere" Werte) und anschließend Rückgabe eines Ergebnisses

Was passiert, wenn die Abbruchbedingung fehlt?

```
public class FactorialOverflow {  
    static int factorial(int n) {  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(5));  
    }  
}
```

StackOverflowError

FactorialOverflow x

"C:\Program Files\AdoptOpenJDK\jdk-11.0.8.10-hotspot\bin\java.exe" "-jav
Exception in thread "main" java.lang.StackOverflowError Create breakpoint
at session09c.FactorialOverflow.factorial(FactorialOverflow.java:6)
at session09c.FactorialOverflow.factorial(FactorialOverflow.java:6)



code22.

Factorial
Overflow

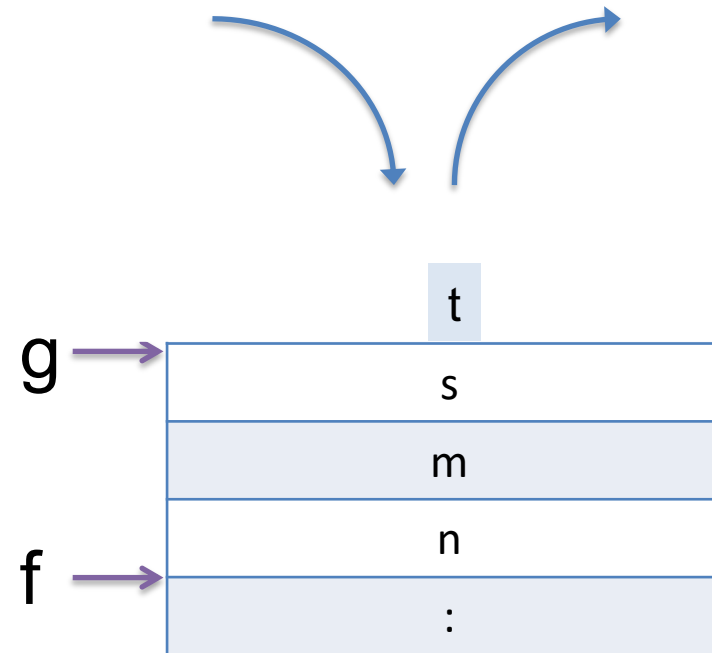
Exkurs: Stack Memory

Kennen wir schon aus der Betrachtung des Java-Speicher-Modell: **Stack Memory**:

- Speicher für den Inhalt der Variablen aus dem aktuellen Kontext ("Scope")
- Wächst für jeden neuen Block und jeden Funktionsaufruf
- Wenn Block/Aufruf beendet, wird der Inhalt, der sich auf den Block/auf die Funktion bezogen hat, wieder gelöscht

Beispiel: Stack-Memory bei Funktionsaufruf

```
void f(int n) {  
    int m = n + 1;  
    int s = n*n;  
    int t = g(m);  
    :  
}
```



Stack
("Stapel"),
wächst
nach
oben

Stack Memory

- begrenzter Speicher
- üblicherweise ausreichend auch für große Rekursionstiefe
- Aber nicht im Beispiel, denn
factorial ruft factorial ruft factorial ruft factorial ... auf
- Stack wächst also immer weiter (immer neue Parameterwerte für n werden dort abgelegt)

Fibonacci-Zahlen

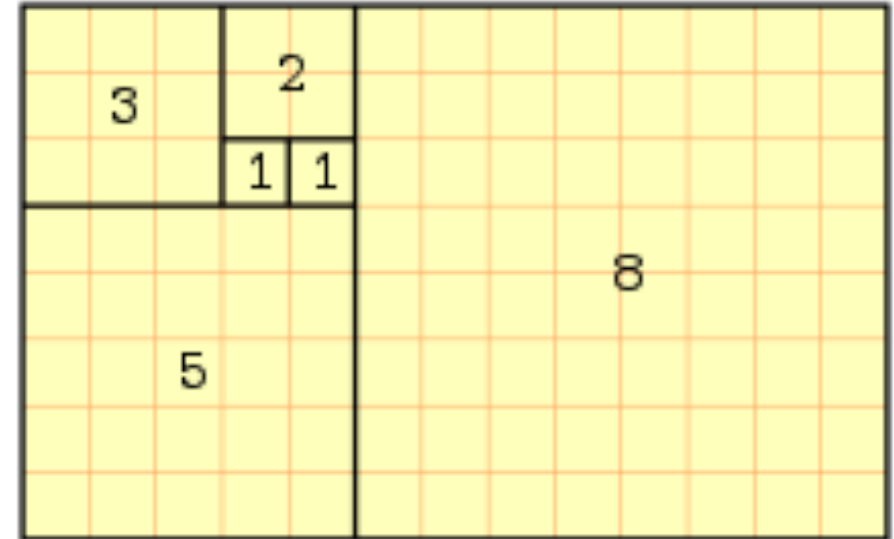
Zahlenfolge benannt nach Leonardo Fibonacci (1170-1240)

$$f_1 = f_2 = 1$$

$$f_n = f_{n-1} + f_{n-2} \text{ für } n > 2$$

"Summe beider Vorgänger"

1, 1, 2, 3, 5, 8, 13, ...
0+1 1+1 1+2 2+3 3+5 5+8



$\frac{f_{n+1}}{f_n}$ nähert sich dem goldenen Schnitt

$$\frac{a+b}{a} = \frac{a}{b}$$

Fibonacci (rekursiv)

$$f_1 = f_2 = 1$$

$$f_n = f_{n-1} + f_{n-2} \text{ für } n > 2$$

Solche rekursiv definierten mathematischen Funktionen lassen sich sehr gut in rekursiven Java-Code umsetzen!

```
public int fibonacci(int n) {  
    if (n == 1 || n == 2) {  
        return 1;  
    }  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```



code22.

FibonacciRec

Mehrfachberechnung

```
public int fibonacci(int n) {  
    if (n == 1 || n == 2) {  
        return 1;  
    }  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

- Überlegen Sie sich, wie oft `fibonacci(3)` bei der Berechnung von `fibonacci(5)` aufgerufen wird.

Mehrfachberechnung

`f := fibonacci`

`f(5)`



(Unnötige!) Doppelberechnung
von Funktionswerten:

<code>f(5)</code>	1 mal
-------------------	-------

Spoileralarm

Fibonacci (iterativ)

Idee: Speichert drei Werte

- vorletzter Wert (secondLast)
- letzter Wert (last)
- aktueller Wert (current)

und schiebt diese immer eins weiter

Dadurch wird jeder Zwischenwert

`fibonacci(i)`, $1 \leq i \leq n$, nur einmal berechnet



code22.

FibonacciIt

```
public static int fibonacci(int n) {  
    if (n == 1 || n == 2) {  
        return 1;  
    }  
    int secondLast = 1;  
    int last = 1;  
    int current = 1;  
    for (int i = 3; i <= n; i++) {  
        current = last + secondLast;  
        secondLast = last;  
        last = current;  
    }  
    return current;  
}
```

Fakultät (iterativ)

```
static int factorial(int n) {  
    // factorial(n) = 1 * 2 * 3 * ... * n  
    int result = 1;  
    for (int i = 1; i <= n; i++) {  
        result = result * i;  
    }  
    return result;  
}
```

- Das hier ist die übliche Implementierung der Methode.
- Man könnte die Schleife auch von n bis 1 rückwärts laufen lassen:
for (int i = n; i >= 1; i--) { ... }



code22.

Factorial
Iterative

Rekursion versus Iteration

- Rekursive Programme sind oft eleganter aufzuschreiben und einfacher zu lesen
- Iterative Programme vermeiden zusätzlichen Speicherbedarf (Call-Stack) und (bei entsprechender Implementierung) Doppelberechnungen

Es gilt (mathematisch bewiesen): Jede Aufgabe, die durch ein rekursives Programm gelöst werden kann, lässt sich auch durch ein iteratives Programm lösen, und umgekehrt.

Aber: Aufwand "rekursiv → iterativ" teilweise sehr hoch

Beispiel: for-Schleife durch Rekursion ersetzen

```
static void printSequence(int from, int to) {  
    for (int i = from; i <= to; i++) {  
        System.out.println(i);  
    }  
}
```

```
static void printSequence(int from, int to) {  
    System.out.println(from);  
    if (from == to) {  
        return;  
    }  
    printSequence(from + 1, to);  
}
```

```
printSequence(1, 10);
```



code22.
ForLoopRec

Exkurs: Funktionale Programmierung

- Funktionen eigenständige Objekte, und man kann Funktionen auf Funktionen anwenden
 - z.B. auch möglich in JavaScript
 - nicht möglich in Java, denn hier kann man Funktionen nur innerhalb einer Klasse definieren und nur Instanzen einer Klasse, nicht jedoch einer Funktion herumreichen.
- **Iteration (Schleifen) werden durch Rekursion ersetzt**
 - Einige Programmiersprachen wie Haskell, Lisp und OCaml verzichten gänzlich auf Schleifen
- Vermeidung von "Zustand", also (Zwischen-)Variablen, von deren Zustand die (globale) Programmsteuerung abhängt
 - funktional programmierte Anwendungen sind leichter zu testen, da man einfach jede Funktion für sich auf Korrektheit überprüfen kann

Imperative Programmierung

- Java ist eine **imperative** Programmiersprache
- Anweisungen werden Schritt für Schritt abgearbeitet, unter Verwendung von Kontrollstrukturen wie Schleife und Bedingung
- lokale und globale Variablen werden verwendet, um den Zustand des Programms zu speichern
- Seit Version 8 enthält Java mit Lambda-Ausdrücken und der Stream-API auch Elemente der Funktionalen Programmierung
 - mehr dazu in Vorlesung Software-Engineering 1

Rekursion: Schwierig?

- Schwierig ist es meistens, sich zu überlegen, wie man rekursiv ein Problem lösen kann.
 - Wenn man sich eine rekursive Lösungsstrategie überlegt hat, ist die Implementierung dann meist sehr einfach.
- Daher: Nicht gleich losprogrammieren, sondern Problem erst (mathematisch-logisch) analysieren.

Beispiel Zweierpotenz

Wollen 2^n *rekursiv* berechnen (für $n \geq 0$).

Überlegung: Wie kann ich die Berechnung von 2^n auf "kleinere" Werte zurückführen?

Idee: $2^n = 2 \cdot 2^{n-1}$, wenn $n > 0$

Denn $2^1 = 2 \cdot 2^0 = 2 \cdot 1 = 2$,

$2^2 = 2 \cdot 2^1 = 4$,

$2^3 = 2 \cdot 2^2 = 8$,

usw.

Und für $n = 0$? Einfach 1 zurückgeben.

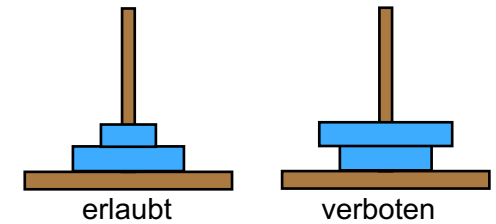
```
static int powerOfTwoTo(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return 2 * powerOfTwoTo(n-1);  
}
```



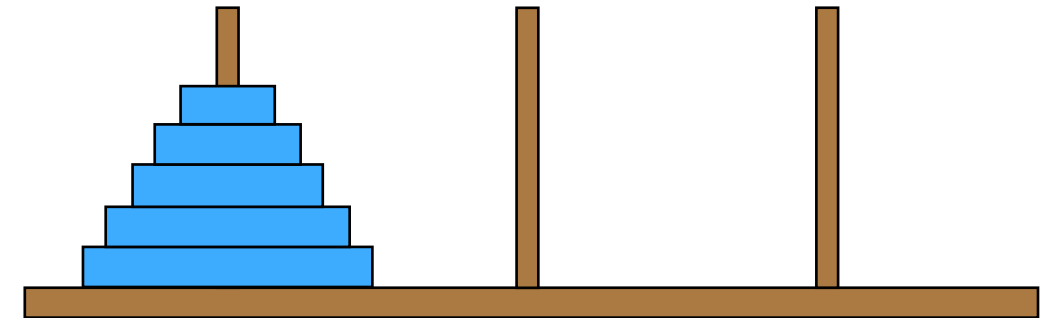
code22.
PowerOfTwo

Beispiel: Lager

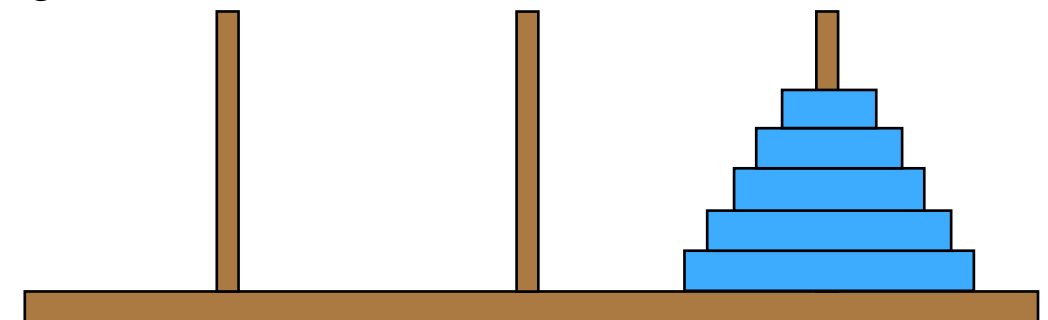
- 3 Läger, Waren mit n unterschiedlichen Mindesthaltbarkeitsdaten (MHD)
- Ware im Lager sortiert nach MHD
 - Leicht verderbliche Ware oben, damit man sie zuerst entnimmt
- Anfangs alle Waren in Lager 1
- Aufgabe: Lagerinhalt von 1 nach 3 bewegen
 - Nur Lager 1, 2, 3 zur Einlagerung benutzbar
 - Transportkapazität **eine Ware je Transportvorgang**
 - Waren in jedem Lager nach MHD sortiert ablegen
- Wie vorgehen?



Ausgangssituation

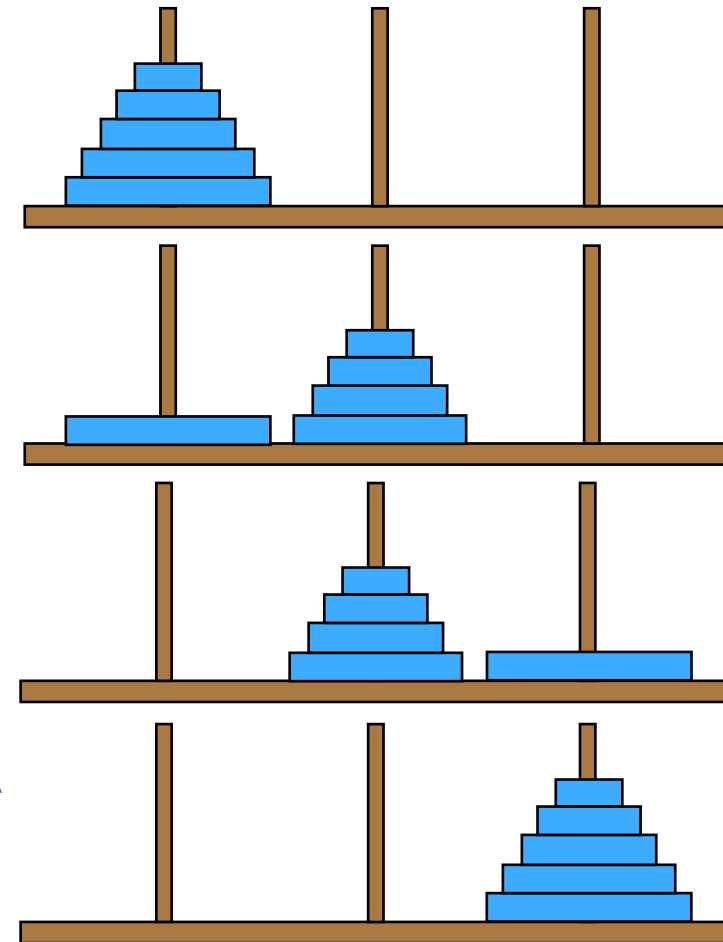


gewünschter Endzustand



Rekursiver Lösungsansatz

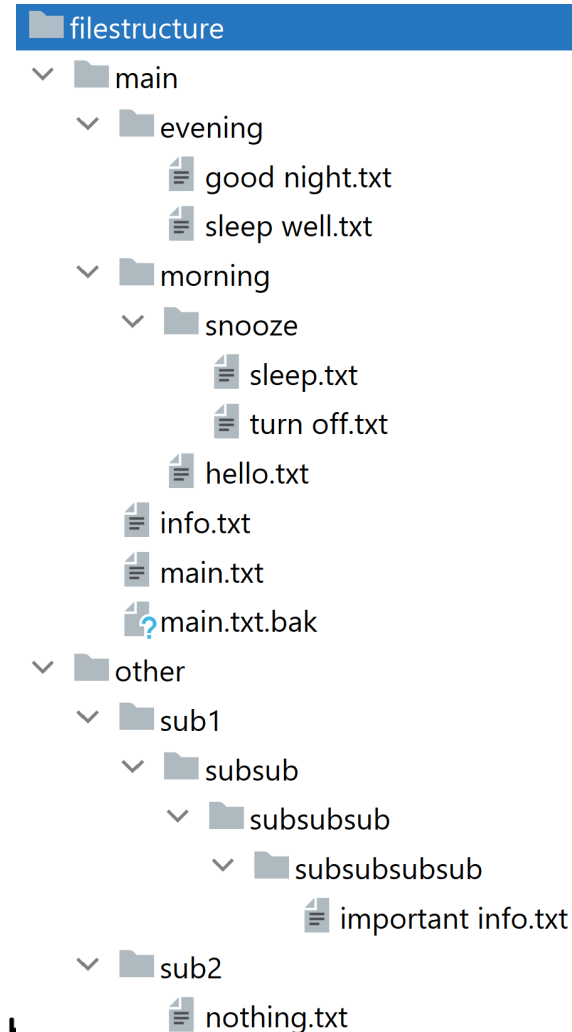
- k Waren von *Start* nach *Ziel* bewegen und dabei *Ausweichlager* verwenden
 - Nur 1 Ware im Lager: Ware von *Start* nach *Ziel*. Fertig.
 - Im Lager Anzahl Waren $k > 1$:
 - Erste $k-1$ Waren von *Start* nach *Ausweichlager*
 - Am längsten haltbare Ware von *Start* nach *Ziel* bewegen
 - $k-1$ Waren von *Ausweichlager* nach *Ziel*
- In Java programmierte Version des "Türme von Hanoi"-Spiels unter <https://github.com/neshanjo/towers-of-hanoi>



code22.

Transport

Beispiel rekursive Datei-Ausgabe



Erwartete Ausgabe (wie "find" in Linux):

```
.\filestructure
.\filestructure\main
.\filestructure\main\evening
.\filestructure\main\evening\good night.txt
.\filestructure\main\evening\sleep well.txt
.\filestructure\main\info.txt
.\filestructure\main\main.txt
.\filestructure\main\main.txt.bak
.\filestructure\main\morning
.\filestructure\main\morning\hello.txt
.\filestructure\main\morning\snooze
.\filestructure\main\morning\snooze\sleep.txt
.\filestructure\main\morning\snooze\turn off.txt
.\filestructure\other
.\filestructure\other\sub1
.\filestructure\other\sub1\subsub
.\filestructure\other\sub1\subsub\subsubsub
.\filestructure\other\sub1\subsub\subsubsub\subsubsubsub
.\filestructure\other\sub1\subsub\subsubsub\
subsubsubsub\important info.txt
.\filestructure\other\sub2
.\filestructure\other\sub2\nothing.txt
```



code22.
ListFiles

 Code ansehen!

Rekursive Datei-Ausgabe

`listRecursively(p):`

Gegeben: Pfad `p`

Gesucht: Ausgabe des Inhalts von `p` inkl. des Inhalts aller Unterverzeichnisse

Lösung:

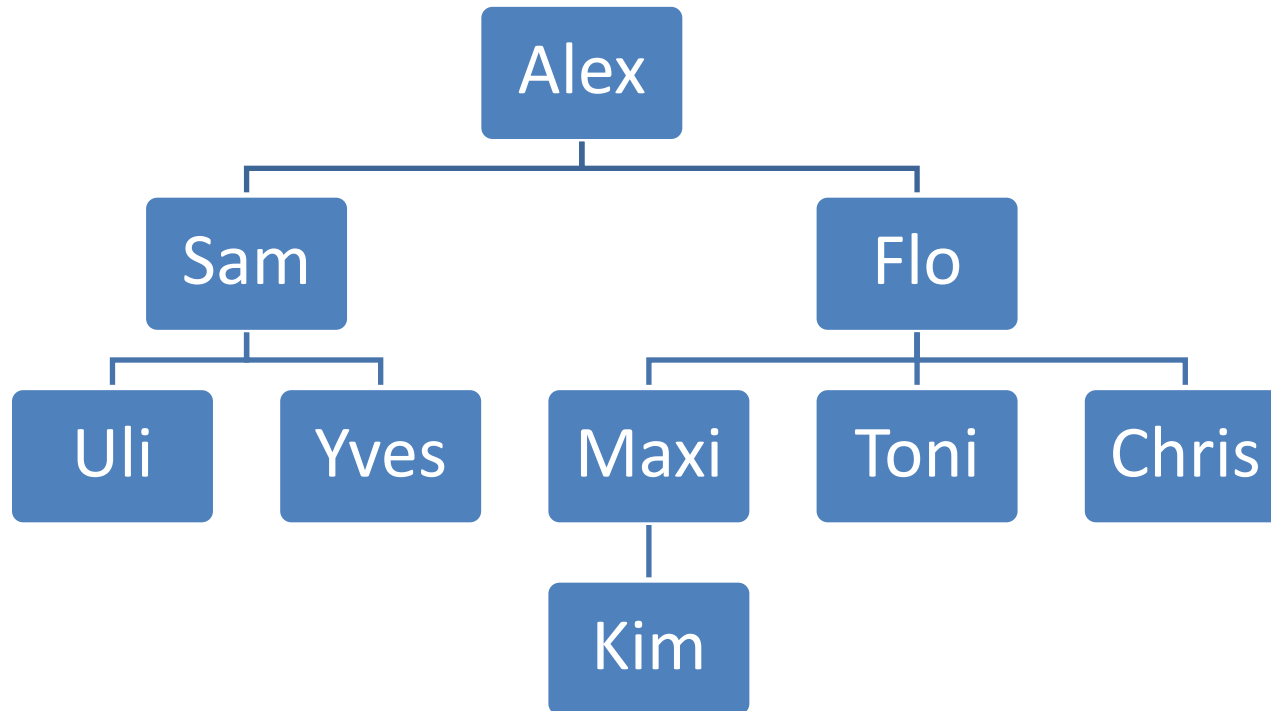
Gib `p` aus.

Falls `p` ein Verzeichnis ist:

Für jeden Pfad `u` in `p`: Wende `listRecursively(u)` an

(Diesen Algorithmus nennt man *Baum-Traversierung in Prä-Order*, vgl. Algorithmen und Datenstrukturen)

Beispiel Ausgabe Hierarchie



Funktioniert mit gleichem Prinzip (Algorithmus) wie Dateiausgabe



Code ansehen!



Employee +
EmployeeMain

Rekursion

- Programmierstil, aber auch wichtiges **Problem-Lösungs-Prinzip**
- Problem auf "kleinere" (Teil-)Probleme zurückführen
 - Fakultät für $n-1$
 - Zweierpotenz für $n-1$
 - Lagerverschiebung für kleineres Lager
 - Dateiausgabe für Unterverzeichnis
- Daraus Lösung berechnen/zusammensetzen
- Immer überprüfen: **Wenn bei einer rekursiven Funktion die Abbruchbedingung (der Rekursionsanker) fehlt, stimmt etwas nicht!**

„Um Rekursion zu verstehen, muss man zuerst
Rekursion verstehen.“

beliebter Informatiker-Witz